DELHI TECHNOLOGICAL UNIVERSITY

Department of Computer Science and Engineering



LAB REPORT

SUBJECT: ADVANCED SYSTEM AND NETWORK SECURITY (AFI601)

Submitted To:
Dr. Ashish Kumar
Ms. Anushri Awasthi
Dept. of Computer
Science and Engineering

Submitted By:
Adarsh Kumar
26/AFI/024

INDEX

Experiment No.	Experiment Title	Page No.	Date	Remarks/ Sign
1	Caesar's Cipher	1	31-08-2026	
2	Playfair Algorithm	3	31-08-2026	
3	Hill Cipher	7	14-09-2026	
4	Multiplicative Inverse under Mod	11	14-09-2026	
5	RSA	13	28-09-2026	
6	Diffie Hellman	16	05-10-2026	
7	DSS using RSA	18	12-10-2026	
8	SHA-512	22	27-10-2026	
9	S-DES	24	27-10-2026	
10	ElGamal Encryption and Decryption	26	27-10-2026	

Experiment 1

Aim

To implement Caesar's Cipher in Java for encryption and decryption of a plaintext message using a given secret key.

Theory

Caesar's Cipher is a classical substitution encryption technique in which each letter of the plaintext is shifted by a fixed number of positions in the alphabet. The number of positions shifted is determined by a secret key.

The algorithm uses the following formulas:

Encryption:

$C = (P + K) \bmod 26$

Decryption:

$P = (C - K + 26) \bmod 26$

Where:

- P represents the numerical position of a plaintext character.
- C represents the numerical position of a ciphertext character.
- K represents the encryption key.
- Mod 26 ensures that the result remains within the 26 letters of the English alphabet.

In this experiment, the plaintext is "Hello" and the key is 10. The program encrypts the message and then decrypts the resulting ciphertext to recover the original plaintext.

The program preserves uppercase and lowercase letters separately. The key is reduced modulo 26 to handle values greater than 26.

Program

```
public class CaeserCipher {
    public static void main(String args[]) {
        String plaintext = "Hello";
        int key = 10;

        String ciphertext = encrypt(plaintext, key);

        System.out.println("Cipher Text: " + ciphertext);
        System.out.println("Plain Text: " + decrypt(ciphertext, key));
    }

    static String encrypt(String pt, int key) {
        key = key%26;
        String ct = "";
        for(int i=0;i<pt.length();i++) {
            char ch = pt.charAt(i);
            if (ch >= 'A' && ch <= 'Z') {
                ct += (char) ('A' + (ch - 'A' + key) % 26);
            } else if (ch >= 'a' && ch <= 'z') {
                ct += (char) ('a' + (ch - 'a' + key) % 26);
            }
```

```
        }

        return ct;
    }

    static String decrypt(String ct, int key) {
        key = key%26;
        String pt = "";
        for(int i=0;i<ct.length();i++) {
            char ch = ct.charAt(i);
            if (ch >= 'A' && ch <= 'Z') {
                pt += (char) ('A' + (ch - 'A' - key + 26) % 26);
            } else if (ch >= 'a' && ch <= 'z') {
                pt += (char) ('a' + (ch - 'a' - key + 26) % 26);
            }
        }

        return pt;
    }
}
```

Output

```
$ java CaeserCipher.java
Cipher Text: Rovvy
Plain Text: Hello
```

Conclusion

Caesar's Cipher was successfully implemented in Java. The plaintext was encrypted using a secret key of 10, and the ciphertext was decrypted to recover the original message. This experiment demonstrates the basic principles of substitution-based encryption and decryption.

Experiment 2

Aim

To implement the Playfair Cipher algorithm in Java for encrypting a plaintext message using a 5 × 5 key matrix.

Theory

The Playfair Cipher is a classical symmetric encryption technique that encrypts pairs of letters instead of individual characters. It uses a 5 × 5 matrix constructed using a secret keyword. In this experiment, a predefined matrix is used.

The matrix used in this experiment is:

K E Y W O

R D A B C

F G H I L

M N P Q S

T U V X Z

The letters I and J are generally treated as the same character because the matrix contains only 25 cells.

Encryption Rules:

1. **Same Row:** Replace each letter with the letter immediately to its right. If a letter is at the end of a row, wrap around to the beginning.
2. **Same Column:** Replace each letter with the letter immediately below it. If a letter is at the bottom, wrap around to the top.
3. **Different Rows and Columns:** Replace each letter with the letter in the same row but in the column occupied by the other letter.

Before encryption, the plaintext is divided into pairs of letters. If a pair contains repeated letters, an 'X' is inserted between them. If the final pair contains only one letter, an 'X' is appended to complete it.

In this experiment, the plaintext is **ADARSHKUMAR**. The program generates letter pairs, determines their positions in the matrix, and applies the encryption rules to produce the ciphertext.

Program

```
import java.util.List;
import java.util.ArrayList;
import java.util.Arrays;

public class PlayfairCipher {
   public static void main(String args[]) {
      String[][] mat = {
         {"K","E","Y","W","O"},
         {"R","D","A","B","C"},
         {"F","G","H","I","L"},
         {"M","N","P","Q","S"},
         {"T","U","V","X","Z"}
```

```
    };

    String plainText = "ADARSHKUMAR";

    String cipherText = genCipher(plainText, mat);

    System.out.println("Cipher Text: " + cipherText);
}

public static String genCipher(String plainText, String[][] mat) {
    List<String> pairs = getPairs(plainText);
    String cipher = "";
    int[][] cords = new int[2][2];
    for(String pair: pairs) {
        cords = getCords(pair, mat);
        // chars in same row
        if(cords[0][1]==cords[1][1]) {
            if(cords[0][0]+1>4) cipher += mat[0][cords[0][1]];
            else cipher += mat[cords[0][0]+1][cords[0][1]];

            if(cords[1][0]+1>4) cipher += mat[0][cords[1][1]];
            else cipher += mat[cords[1][0]+1][cords[1][1]];
        }
        // chars in same column
        else if(cords[0][0]==cords[1][0]){
            if(cords[0][1]+1>4) cipher += mat[cords[0][0]][0];
            else cipher += mat[cords[0][0]][cords[0][1]+1];

            if(cords[1][1]+1>4) cipher += mat[cords[1][0]][0];
            else cipher += mat[cords[1][0]][cords[1][1]+1];
        }
        // chars not in same row or column
        else {
            cipher += mat[cords[0][0]][cords[1][1]];
            cipher += mat[cords[1][0]][cords[0][1]];
        }
    }

    return cipher;
}

public static int[][] getCords(String pair, String[][] mat) {
    int[][] cords = new int[2][2];
    for(int i=0;i<mat.length;i++) {
        for(int j=0;j<mat[0].length;j++) {
            if(pair.substring(0,1).equals("J")) {
                cords[0][0] = 2;
                cords[0][1] = 3;
            }
            else if(mat[i][j].equals(pair.substring(0,1))) {
```

```
                cords[0][0] = i;
                cords[0][1] = j;
            }
            if(pair.substring(1,2).equals("J")) {
                cords[1][0] = 2;
                cords[1][1] = 3;
            }
            else if(mat[i][j].equals(pair.substring(1,2))) {
                cords[1][0] = i;
                cords[1][1] = j;
            }
        }
    }

    return cords;
}

public static List<String> getPairs(String plainText) {
    List<String> pairs = new ArrayList<>();
    int i = 0;
    String padding = "X";
    String s = "";
    while(i<plainText.length()) {
        s = "";
        if(i==plainText.length()-1) {
            s += plainText.charAt(i);
            s += padding;
            i += 2;
        }
        else if(i+1<plainText.length() && plainText.charAt(i)==plainText.charAt(i+1)) {
            s += plainText.charAt(i);
            s += padding;
            i++;
        }
        else {
            s += plainText.charAt(i);
            s += plainText.charAt(i+1);
            i += 2;
        }
        pairs.add(s);
    }

    return pairs;
}
}
```

Output

```
$ java PlayfairCipher.java
Cipher Text: BABDPLETPRBT
```

Conclusion

The Playfair Cipher algorithm was implemented in Java to encrypt a plaintext message using a 5 × 5 matrix. The program divides the plaintext into pairs of letters and applies the appropriate encryption rules to generate the ciphertext. This experiment demonstrates pair-based substitution encryption.

Experiment 3

Aim

To implement the Hill Cipher algorithm in Java for encrypting and decrypting a message using matrix multiplication and modular arithmetic.

Theory

The Hill Cipher is a classical symmetric encryption technique that uses linear algebra to encrypt groups of letters. It operates on blocks of plaintext characters and uses a square key matrix for encryption and decryption.

Each letter is converted into a numerical value between 0 and 25, where A = 0, B = 1, ..., Z = 25.

Encryption Formula:

$C = K \times P \bmod 26$

Decryption Formula:

$P = K^{-1} \times C \bmod 26$

Where:

- C represents the ciphertext vector.
- P represents the plaintext vector.
- K represents the key matrix.
- K^{-1} represents the modular inverse of the key matrix.
- mod 26 ensures that the resulting values remain within the alphabet range.

In this experiment, the plaintext is **ACT**, and the key is **GYBNQKURP**. The key is arranged into a 3 × 3 matrix:

G Y B

N Q K

U R P

The plaintext is converted into a column vector and multiplied by the key matrix. The resulting values are reduced modulo 26 and converted back into letters to generate the ciphertext.

For decryption, the program calculates the determinant, modular inverse, and adjoint of the key matrix to obtain its modular inverse. The inverse matrix is then multiplied by the ciphertext vector to recover the original plaintext.

Program

```
import java.util.Arrays;

// Ascii of A-Z is 65-90
// Ascii of a-z is 97-122

public class HillCipher {
    public static void main(String[] args) {
        String PT = "ACT";
        String key = "GYBNQKURP";

        String CT = getCipherText(key, PT);
```

```
        System.out.println("The Cipher Text is " + CT);
        System.out.println("The Plain Text is " + getPlainText(key, CT));
    }

    static String getCipherText(String key, String PT) {
        int[][] keyMat = getMat(key, key.length()/PT.length(), PT.length());
        int[][] ptMat = getMat(PT, PT.length(), 1);
        String CT = "";

        char[][] mult = matMult(keyMat, ptMat, keyMat.length, ptMat.length,
ptMat[0].length);

        for(int i=0;i<mult.length;i++) {
            for(int j=0;j<mult[0].length;j++) {
                CT += mult[i][j];
            }
        }

        return CT;
    }

    static int[][] getMat(String str, int x, int y) {
        int[][] mat = new int[x][y];

        int count = 0;
        for(int i=0;i<x;i++) {
            for(int j=0;j<y;j++) {
                mat[i][j] = (int) str.charAt(count) - (int) 'A';
                count++;
            }
        }

        return mat;
    }

    static char[][] matMult(int[][] mat1, int[][] mat2, int n, int m, int p) {
        int[][] res = new int[n][p];
        char[][] charMat = new char[n][p];

        for(int i=0;i<n;i++) {
            for(int j=0;j<p;j++) {
                for(int k=0;k<m;k++) {
                    res[i][j] += mat1[i][k] * mat2[k][j];
                }
                charMat[i][j] = (char) (res[i][j] % 26 + (int) 'A');
            }
        }

        return charMat;
    }

    static int[][] getInverse(int[][] mat, int n) {
        int[][] adj = new int[n][n];
        int det = getDeterminant(mat, n);
        int invDet = modInverse(det, 26);

        if(det == 0) {
            System.out.println("Inverse doesn't exist");
            return null;
        }

        adj = getAdjoint(mat, n);

        for(int i=0;i<n;i++) {
```

```
            for(int j=0;j<n;j++) {
                adj[i][j] = (adj[i][j] * invDet) % 26;
                if(adj[i][j] < 0) adj[i][j] += 26;
            }
        }

        return adj;
    }

    static int getDeterminant(int[][] mat, int n) {
        int det = 0;
        if(n == 1) return mat[0][0];
        if(n == 2) return (mat[0][0] * mat[1][1] - mat[0][1] * mat[1][0]);

        for(int i=0;i<n;i++) {
            int[][] temp = new int[n-1][n-1];
            for(int j=1;j<n;j++) {
                for(int k=0;k<n;k++) {
                    if(k < i) temp[j-1][k] = mat[j][k];
                    else if(k > i) temp[j-1][k-1] = mat[j][k];
                }
            }
            det += (i%2==0 ? 1 : -1) * mat[0][i] * getDeterminant(temp, n-1);
        }

        return det;
    }

    static int modInverse(int a, int m) {
        a = a % m;
        for(int x=1;x<m;x++) {
            if((a * x) % m == 1) return x;
        }
        return 1;
    }

    static int[][] getAdjoint(int[][] mat, int n) {
        int[][] adj = new int[n][n];
        if(n == 1) {
            adj[0][0] = 1;
            return adj;
        }

        for(int i=0;i<n;i++) {
            for(int j=0;j<n;j++) {
                int[][] temp = new int[n-1][n-1];
                for(int row=0;row<n;row++) {
                    for(int col=0;col<n;col++) {
                        if(row != i && col != j) {
                            int r = row < i ? row : row - 1;
                            int c = col < j ? col : col - 1;
                            temp[r][c] = mat[row][col];
                        }
                    }
                }
                adj[j][i] = (int) Math.pow(-1, i+j) * getDeterminant(temp, n-
1);
            }
        }

        return adj;
    }

    static String getPlainText(String key, String CT) {
```

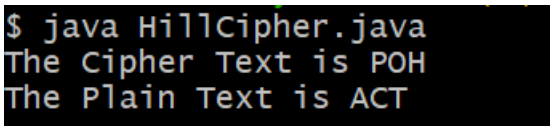
```
        int[][] keyMat = getMat(key, key.length()/CT.length(), CT.length());
        int[][] ctMat = getMat(CT, CT.length(), 1);
        String PT = "";

        int[][] invKeyMat = getInverse(keyMat, keyMat.length);
        char[][] mult = matMult(invKeyMat, ctMat, invKeyMat.length,
ctMat.length, ctMat[0].length);

        for(int i=0;i<mult.length;i++) {
            for(int j=0;j<mult[0].length;j++) {
                PT += mult[i][j];
            }
        }

        return PT;
    }
}
```

Output



Conclusion

The Hill Cipher algorithm was implemented in Java using matrix multiplication and modular arithmetic. The plaintext was encrypted using a 3 × 3 key matrix, and the ciphertext was decrypted using the inverse key matrix to recover the original message. This experiment demonstrates the application of linear algebra in classical cryptography.

Experiment 4

Aim

To implement a Java program to find the multiplicative inverse of an integer under a given modulo.

Theory

The modular multiplicative inverse of an integer *a* under modulo *m* is an integer *x* that satisfies the following condition:

(a × x) mod m = 1

The multiplicative inverse exists only when the greatest common divisor of *a* and *m* is 1, i.e., gcd(a, m) = 1.

In this experiment, the values are:

- **a = 3**
- **m = 26**

The program searches for an integer *x* from 1 to *m − 1*. For each value, it checks whether (a × x) mod m equals 1. When the condition is satisfied, the value is returned as the modular inverse.

For the given values:

(3 × 9) mod 26 = 27 mod 26 = 1

Therefore, the multiplicative inverse of 3 modulo 26 is **9**.

Modular inverses are widely used in cryptographic algorithms, including the RSA algorithm and classical encryption techniques.

Program

```
public class ModInverse {
    public static void main(String args[]) {
        int a = 3;
        int m = 26;
        System.out.println("Modular Inverse of " + a + " under modulo " + m + "
is " + modInverse(a, m));
    }

    static int modInverse(int a, int m) {
        a = a % m;
        for(int x=1;x<m;x++) {
            if((a * x) % m == 1) return x;
        }
        return 1;
    }
}
```

Output

```
$ java ModInverse.java
Modular Inverse of 3 under modulo 26 is 9
```

Conclusion

The Java program successfully calculated the multiplicative inverse of 3 under modulo 26. The result was verified using the modular arithmetic condition $(3 \times 9) \bmod 26 = 1$. This experiment demonstrates the importance of modular inverses in cryptographic computations.

Experiment 5

Aim

To implement the RSA public-key cryptographic algorithm in Java for generating public and private keys and performing encryption and decryption of a message.

Theory

RSA is an asymmetric cryptographic algorithm that uses two different keys: a public key for encryption and a private key for decryption. Its security relies on the computational difficulty of factoring large integers into their prime factors.

Steps of the RSA Algorithm

1. Key Generation

- Select two prime numbers, p and q.
- Calculate the modulus: $\mathbf{N = p \times q}$
- Calculate Euler's totient: $\mathbf{\varphi(N) = (p - 1)(q - 1)}$
- Choose a public exponent e such that $\mathbf{gcd(e, \varphi(N)) = 1}$.
- Calculate the private exponent d such that $\mathbf{(e \times d) \bmod \varphi(N) = 1}$.

The public key is **(e, N)**, and the private key is **(d, N)**.

2. Encryption

The plaintext message M is encrypted using the public key:

$$\mathbf{C = M^e \bmod N}$$

3. Decryption

The ciphertext C is decrypted using the private key:

$$\mathbf{M = C^d \bmod N}$$

Where:

- p and q are prime numbers.
- N is the modulus.
- $\varphi(N)$ is Euler's totient function.
- e is the public exponent.
- d is the private exponent.
- M is the original message.
- C is the encrypted message.

In this experiment, the program accepts two prime numbers from the user, calculates the modulus and totient, generates the public and private exponents, and performs encryption and decryption on a numerical message.

Program

```
import java.util.Arrays;
import java.util.Scanner;

public class RSA {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("==== KEY GENERATION ====");
        int p = get("Enter P: ", sc);
        int q = get("Enter Q: ", sc);

        int N = p*q;
        int phi_n = (p-1) * (q-1);

        int[] keys = keyGeneration(phi_n);
        System.out.println("Public Key: " + keys[0]);
        System.out.println("Private Key: " + keys[1]);

        System.out.println();
        System.out.println("==== Encryption ====");
        System.out.print("Enter the number to be encrypted (0 ≤ message < " + N
+ "): ");
        int message = sc.nextInt();
        int encrypted_message = (int) (Math.pow(message, keys[0]) % N);
        System.out.println("Encrypted message: " + encrypted_message);

        System.out.println();
        System.out.println("==== Decryption ====");
        int decrypted_message = (int) (Math.pow(encrypted_message, keys[1]) %
N);
        System.out.println("Decrypted message: " + decrypted_message);
    }

    static int get(String s, Scanner sc) {
        System.out.print(s);
        int x = sc.nextInt();
        return x;
    }

    // generating public key and private key
    static int[] keyGeneration(int phi_n) {
        int public_key = getCoprime(phi_n);
        int private_key = modInverse(public_key, phi_n);
        return new int[] {public_key, private_key};
    }

    static int modInverse(int a, int m) {
        a = a % m;
        for(int x=1;x<m;x++) {
            if((a * x) % m == 1) return x;
        }
        return 1;
    }

    // Euclidean Algorithm to get gcd
    public static int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    // Returns true if the numbers are coprime, false otherwise
    public static int getCoprime(int phi_n) {
        for(int i=2;i<phi_n;i++) {
            if(gcd(i, phi_n) == 1) return i;
        }

        return 1;
    }
}
```

Output

```
$ java RSA.java
==== KEY GENERATION ====
Enter P: 5
Enter Q: 3
Public Key: 3
Private Key: 3

==== Encryption ====
Enter the number to be encrypted (0 ≤ message < 15): 10
Encrypted message: 10

==== Decryption ====
Decrypted message: 10
```

Conclusion

The RSA algorithm was implemented in Java to generate public and private keys and perform message encryption and decryption. The original message was successfully recovered after decryption, demonstrating the basic working principle of asymmetric cryptography.

Experiment 6

Aim

To implement the Diffie-Hellman key exchange algorithm in Java to establish a shared secret key between two parties over an insecure communication channel.

Theory

The Diffie-Hellman key exchange is a cryptographic technique that allows two parties to establish a common secret key over a public channel without directly transmitting the secret key.

The algorithm uses a prime number *p* and a generator *g*, which are publicly known. Alice and Bob independently choose their private keys and exchange their public keys to calculate a common secret.

Steps of the Algorithm

1. Both parties agree on a prime number *p* and a generator *g*.
2. Alice selects a private key *a*, and Bob selects a private key *b*.
3. Alice calculates her public key:

$$A = g^a \bmod p$$

4. Bob calculates his public key:

$$B = g^b \bmod p$$

5. Alice calculates the shared secret using Bob's public key:

$$K_1 = B^a \bmod p$$

6. Bob calculates the shared secret using Alice's public key:

$$K_2 = A^b \bmod p$$

Both parties obtain the same secret because:

$$K_1 = K_2 = g^{ab} \bmod p$$

In this experiment, the Java program accepts the prime number, generator, and private keys of Alice and Bob as input. It calculates their public keys and independently derives the shared secret key for each party. Finally, it checks whether both secret keys are equal.

Program

```
import java.util.Scanner;

public class DiffieHellman {
    public static void main(String args[]) {
        int p = get("Enter prime number p: ");
        int g = get("Enter generator g: ");

        int a = get("Enter private key of alice: ");
        int b = get("Enter private key of bob: ");

        // Generating Public Keys
        int A = getMod(g, a, p);
        int B = getMod(g, b, p);

        System.out.println("Public Key A: " + A);
        System.out.println("Public Key B: " + B);

        // Generating Secret Key
```

```
        int secretKey1 = getMod(B, a, p); // Alice generating secret key using
Bob's public key
        int secretKey2 = getMod(A, b, p); // Bob generating secret key using
Alice's public key

        System.out.println("Alice has Secret key 1: " + secretKey1);
        System.out.println("Bob has Secret key 2: " + secretKey2);
        System.out.println(secretKey1==secretKey2);
    }

    static int get(String s) {
        Scanner sc = new Scanner(System.in);

        System.out.print(s);
        int x = sc.nextInt();
        return x;
    }

    static int getMod(int base, int exp, int MOD) {
        long result = 1;
        long b = base % MOD;

        while (exp > 0) {
            if ((exp & 1) == 1) {
                result = (result * b) % MOD;
            }

            b = (b * b) % MOD;
            exp >>= 1;
        }

        return (int) result;
    }

}
```

Output

```
$ java DiffieHellman.java
Enter prime number p: 5
Enter generator g: 3
Enter private key of alice: 10
Enter private key of bob: 20
Public Key A: 4
Public Key B: 1
Alice has Secret key 1: 1
Bob has Secret key 2: 1
```

Conclusion

The Diffie-Hellman key exchange algorithm was successfully implemented in Java. Alice and Bob generated their public keys and independently calculated the same shared secret key without directly exchanging their private keys. This experiment demonstrates the fundamental principle of secure key establishment using public-key cryptography.

Experiment 7

Aim

To implement a digital signature scheme using RSA in Java to generate and verify digital signatures and detect message tampering.

Theory

A digital signature is a cryptographic technique used to verify the authenticity and integrity of a digital message. It helps the receiver confirm that a message was signed using the sender's private key and that the message has not been modified.

In this experiment, RSA is used to create and verify digital signatures. Although the experiment title refers to the Digital Signature Standard (DSS), the implementation uses an **RSA-based digital signature scheme**, rather than the standardized DSA algorithm.

Steps of the Algorithm

1. Key Generation

- Select two prime numbers, p and q.
- Calculate the modulus: $\mathbf{N = p \times q}$.
- Calculate Euler's totient: $\mathbf{\varphi(N) = (p - 1)(q - 1)}$.
- Choose a public exponent e such that gcd(e, φ(N)) = 1.
- Calculate the private exponent d such that $\mathbf{(e \times d) \bmod \varphi(N) = 1}$.

The public key is (e, N), and the private key is (d, N).

2. Digital Signature Generation

The sender calculates a hash value of the message and signs it using the private key.

$$\mathbf{S = H^d \bmod N}$$

Where:

- H is the hash value of the message.
- d is the private exponent.
- S is the digital signature.

3. Signature Verification

The receiver uses the sender's public key to recover the hash value from the signature.

$$\mathbf{H' = S^e \bmod N}$$

The receiver independently calculates the hash of the received message and compares it with the recovered hash.

- If $H' = H$, the signature is considered valid.
- If $H' \neq H$, the signature is considered invalid.

The program also modifies the original message by adding the prefix "Hacked: " and checks whether the altered message passes verification.

The implementation uses a simple custom hash function for educational purposes. It demonstrates the basic concept of signing and verification but is not suitable for real-world cryptographic security.

Program

```
// Digital Signature Standard (DSS) using RSA based signature schema

import java.util.Scanner;

public class DSSUsingRSA {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        System.out.println("==== KEY GENERATION ====");
        int p = get("Enter P: ", sc);
        int q = get("Enter Q: ", sc);

        int N = p * q;
        int phi_n = (p - 1) * (q - 1);

        int[] keys = keyGeneration(phi_n);
        int public_key = keys[0];  // e - published, used by anyone to verify
        int private_key = keys[1]; // d - kept secret, used by the signer only
        System.out.println("Public Key  (e, N): (" + public_key + ", " + N +
")");
        System.out.println("Private Key (d, N): (" + private_key + ", " + N +
")");

        System.out.println();
        System.out.println("==== SIGNING (Sender) ====");
        System.out.print("Enter the message to be signed: ");
        sc.nextLine(); // consume the newline left behind by nextInt()
        String message = sc.nextLine();

        // The message itself is never signed, only its hash
        int hash = hash(message, N);
        int signature = modPow(hash, private_key, N); // S = H^d mod N
        System.out.println("Message: " + message);
        System.out.println("Hash (H): " + hash);
        System.out.println("Signature (S = H^d mod N): " + signature);
        System.out.println("Sender transmits the pair (message, signature)");

        System.out.println();
        System.out.println("==== VERIFICATION (Receiver) ====");
        int recovered_hash = modPow(signature, public_key, N); // H' = S^e mod
N
        int receiver_hash = hash(message, N);
        System.out.println("Hash of received message (H): " + receiver_hash);
        System.out.println("Hash recovered from signature (H' = S^e mod N): " +
recovered_hash);
        System.out.println("Signature is " + (recovered_hash == receiver_hash
                ? "VALID - message is authentic"
                : "INVALID - message or signature was altered"));

        System.out.println();
        System.out.println("==== VERIFICATION OF A TAMPERED MESSAGE ====");
        String tampered_message = "Hacked: " + message;
        System.out.println("Tampered message: " + tampered_message);
        System.out.println("Signature is " + (modPow(signature, public_key, N)
== hash(tampered_message, N)
                ? "VALID - message is authentic"
                : "INVALID - message or signature was altered"));

        sc.close();
    }

    static int get(String s, Scanner sc) {
```

```
        System.out.print(s);
        int x = sc.nextInt();
        return x;
    }

    // generating public key and private key
    static int[] keyGeneration(int phi_n) {
        int public_key = getCoprime(phi_n);
        int private_key = modInverse(public_key, phi_n);
        return new int[] {public_key, private_key};
    }

    // Extended Euclidean Algorithm to find d such that (a * d) % m == 1
    static int modInverse(int a, int m) {
        long old_r = a, r = m;
        long old_s = 1, s = 0;

        while (r != 0) {
            long quotient = old_r / r;

            long temp = old_r - quotient * r;
            old_r = r;
            r = temp;

            temp = old_s - quotient * s;
            old_s = s;
            s = temp;
        }

        if (old_r != 1) return 1; // a and m are not coprime, inverse does not
exist
        return (int) ((old_s % m + m) % m);
    }

    // Euclidean Algorithm to get gcd
    public static int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    // Returns the smallest number coprime to phi_n, used as the public
exponent
    public static int getCoprime(int phi_n) {
        for(int i=2;i<phi_n;i++) {
            if(gcd(i, phi_n) == 1) return i;
        }

        return 1;
    }

    // Hash of the message, reduced modulo N so that it fits the RSA modulus
    static int hash(String message, int N) {
        int h = 7; // non zero seed so that even an empty message hashes to a
valid value
        for (int i = 0; i < message.length(); i++) {
            h = (h * 31 + message.charAt(i)) % N;
        }
        return h;
    }

    // Fast modular exponentiation, (base ^ exponent) % mod
    static int modPow(int base, int exponent, int mod) {
        long result = 1;
        long b = base % mod;
```

```
        while (exponent > 0) {
            if (exponent % 2 == 1) result = (result * b) % mod;
            b = (b * b) % mod;
            exponent /= 2;
        }

        return (int) result;
    }
}
```

Output

```
$ java DSSUsingRSA.java
==== KEY GENERATION ====
Enter P: 5
Enter Q: 7
Public Key  (e, N): (5, 35)
Private Key (d, N): (5, 35)

==== SIGNING (Sender) ====
Enter the message to be signed: 10
Message: 10
Hash (H): 34
Signature (S = H^d mod N): 34
Sender transmits the pair (message, signature)

==== VERIFICATION (Receiver) ====
Hash of received message (H): 34
Hash recovered from signature (H' = S^e mod N): 34
Signature is VALID - message is authentic

==== VERIFICATION OF A TAMPERED MESSAGE ====
Tampered message: Hacked: 10
Signature is INVALID - message or signature was altered
```

Conclusion

The RSA-based digital signature scheme was implemented in Java to demonstrate digital signature generation and verification. The program generates a public-private key pair, signs a message using the private key, and verifies the signature using the public key. It also demonstrates how modifying a message can cause signature verification to fail. This experiment illustrates the role of digital signatures in message authentication and integrity verification.

Experiment 8

Aim

To implement the SHA-512 cryptographic hash function in Java to generate a 512-bit hash value for a given input message.

Theory

SHA-512 (Secure Hash Algorithm 512) is a cryptographic hash function belonging to the SHA-2 family. It accepts an input message of arbitrary length and produces a fixed-size hash value of 512 bits (64 bytes), typically represented as 128 hexadecimal characters.

SHA-512 provides the following properties:

- **Fixed-length output:** Produces a 512-bit hash regardless of the input length.
- **Deterministic:** The same input always produces the same hash.
- **One-way property:** It is computationally infeasible to recover the original message from its hash.
- **Avalanche effect:** A small change in the input can significantly change the resulting hash.

SHA-512 processes the message in 1024-bit blocks using padding, message scheduling, and multiple rounds of bitwise operations.

In Java, the MessageDigest class from the java.security package provides a built-in implementation of SHA-512.

Program

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Scanner;

public class SHA512 {
    public static void main(String[] args)
            throws NoSuchAlgorithmException {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the message: ");
        String message = sc.nextLine();

        MessageDigest md = MessageDigest.getInstance("SHA-512");
        byte[] hash = md.digest(message.getBytes());

        StringBuilder hex = new StringBuilder();

        for (byte b : hash) {
            hex.append(String.format("%02x", b & 0xff));
        }

        System.out.println("SHA-512 Hash: " + hex);

        sc.close();
    }
}
```

Output

```
$ java SHA512.java
Enter the message: Hello
SHA-512 Hash: 3615f80c9d293ed7402687f94b22d58e529b8cc7916f8fac7fddf7
fbd5af4cf777d3d795a7a00a16bf7e7f3fb9561ee9baae480da9fe7a18769e71886b
03f315
```

Conclusion

The SHA-512 hash function was successfully implemented in Java using the MessageDigest class. The program generates a fixed-length 512-bit hash for the given message. This experiment demonstrates the use of cryptographic hashing for data integrity and secure message verification.

Experiment 9

Aim

To implement the subkey generation process of Simplified Data Encryption Standard (S-DES) in Java to generate two 8-bit subkeys from a 10-bit input key.

Theory

Simplified Data Encryption Standard (S-DES) is a simplified version of the DES symmetric-key encryption algorithm. It is commonly used for educational purposes to demonstrate the basic principles of block cipher design.

S-DES uses a 10-bit key to generate two 8-bit subkeys, K1 and K2, which are used in its encryption and decryption processes.

The subkey generation process involves the following steps:

1. **P10 Permutation:** Rearranges the bits of the original 10-bit key according to a predefined permutation.
2. **Splitting:** Divides the permuted key into two 5-bit halves.
3. **Left Shift:** Circularly shifts each half left by one position.
4. **P8 Permutation:** Selects and rearranges eight bits to generate the first subkey, K1.
5. **Second Left Shift:** Circularly shifts both halves left by two additional positions.
6. **P8 Permutation:** Selects and rearranges eight bits to generate the second subkey, K2.

The permutations used are:

- **P10:** 3 5 2 7 4 10 1 9 8 6
- **P8:** 6 3 7 4 8 5 10 9

Program

```
import java.util.Scanner;

public class SDESKeyGeneration {

    static int[] P10 = {3, 5, 2, 7, 4, 10, 1, 9, 8, 6};
    static int[] P8 = {6, 3, 7, 4, 8, 5, 10, 9};

    static String permute(String key, int[] table) {
        StringBuilder result = new StringBuilder();

        for (int position : table) {
            result.append(key.charAt(position - 1));
        }

        return result.toString();
    }

    static String leftShift(String bits, int count) {
        return bits.substring(count) + bits.substring(0, count);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter 10-bit key: ");
        String key = sc.nextLine();

        if (!key.matches("[01]{10}")) {
```

```
            System.out.println("Invalid key. Enter exactly 10 binary bits.");
            sc.close();
            return;
        }

        String permutedKey = permute(key, P10);

        String left = permutedKey.substring(0, 5);
        String right = permutedKey.substring(5);

        left = leftShift(left, 1);
        right = leftShift(right, 1);

        String k1 = permute(left + right, P8);

        left = leftShift(left, 2);
        right = leftShift(right, 2);

        String k2 = permute(left + right, P8);

        System.out.println("Subkey K1: " + k1);
        System.out.println("Subkey K2: " + k2);

        sc.close();
    }
}
```

Output

```
$ java SDESKeyGeneration.java
Enter 10-bit key: 1010000010
Subkey K1: 10100100
Subkey K2: 01000011
```

Conclusion

The S-DES subkey generation algorithm was successfully implemented in Java. The program applies P10 permutation, circular left shifts, and P8 permutation to generate two 8-bit subkeys from a 10-bit key. These subkeys are used in the subsequent stages of S-DES encryption and decryption.

Experiment 10

Aim

To implement the ElGamal cryptographic algorithm in Java, including key generation, encryption, and decryption of a numerical message.

Theory

ElGamal is an asymmetric cryptographic algorithm based on the mathematical difficulty of solving the discrete logarithm problem. It uses a public key for encryption and a private key for decryption.
The algorithm consists of three main stages.

1. Key Generation

- Choose a prime number *p* and a generator *g*.
- Select a private key *x*, where $1 \leq x \leq p - 2$.
- Calculate the public component:

$y = g^x \bmod p$
The public key is *(p, g, y)*, while the private key is *x*.

2. Encryption

- Select a random ephemeral key *k*, where $1 \leq k \leq p - 2$.
- Calculate:

$C1 = g^k \bmod p$
$C2 = M \times y^k \bmod p$
Here, *M* is the numerical plaintext message, which must satisfy $0 \leq M < p$.
The ciphertext is the pair *(C1, C2)*.

3. Decryption
The receiver calculates:
$s = C1^x \bmod p$
The original message is recovered using:
$M = C2 \times s^{-1} \bmod p$

Here, s^{-1} is the multiplicative inverse of *s* modulo *p*.
In this experiment, the program generates the public and private keys, encrypts a numerical message, and decrypts the resulting ciphertext to recover the original message.

Program

```
import java.util.Scanner;
import java.security.SecureRandom;

public class ElGamal {

    static long modPow(long base, long exp, long mod) {
        long result = 1;
        base %= mod;

        while (exp > 0) {
            if ((exp & 1) == 1) {
                result = result * base % mod;
            }
```

```
            base = base * base % mod;
            exp >>= 1;
        }

        return result;
    }

    static long modInverse(long a, long p) {
        long t = 0, newT = 1;
        long r = p, newR = a % p;

        while (newR != 0) {
            long q = r / newR;

            long temp = t - q * newT;
            t = newT;
            newT = temp;

            temp = r - q * newR;
            r = newR;
            newR = temp;
        }

        if (r != 1) {
            throw new ArithmeticException("Inverse does not exist");
        }

        return (t % p + p) % p;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SecureRandom random = new SecureRandom();

        System.out.print("Enter prime number p: ");
        long p = sc.nextLong();

        System.out.print("Enter generator g: ");
        long g = sc.nextLong();

        System.out.print("Enter private key x: ");
        long x = sc.nextLong();

        if (p <= 2 || g <= 1 || g >= p || x < 1 || x > p - 2) {
            System.out.println("Invalid parameters.");
            sc.close();
            return;
        }

        long y = modPow(g, x, p);

        System.out.println("Public Key (p, g, y): ("
                + p + ", " + g + ", " + y + ")");
        System.out.println("Private Key x: " + x);

        System.out.print("Enter message M (0 <= M < p): ");
        long m = sc.nextLong();

        if (m < 0 || m >= p) {
            System.out.println("Invalid message.");
            sc.close();
            return;
        }
```

```
        long k = 1 + random.nextInt((int) (p - 2));

        long c1 = modPow(g, k, p);
        long c2 = (m * modPow(y, k, p)) % p;

        System.out.println("Ciphertext C1: " + c1);
        System.out.println("Ciphertext C2: " + c2);

        long sharedSecret = modPow(c1, x, p);
        long inverse = modInverse(sharedSecret, p);

        long decrypted = (c2 * inverse) % p;

        System.out.println("Decrypted message: " + decrypted);

        sc.close();
    }
}
```

Output

```
$ java ElGamal.java
Enter prime number p: 23
Enter generator g: 5
Enter private key x: 6
Public Key (p, g, y): (23, 5, 8)
Private Key x: 6
Enter message M (0 <= M < p): 10
Ciphertext C1: 11
Ciphertext C2: 21
Decrypted message: 10
```

Conclusion

The ElGamal encryption and decryption algorithm was implemented in Java. The program generates a public-private key pair, encrypts a numerical message using an ephemeral key, and decrypts the ciphertext using the private key. The original message is recovered successfully, demonstrating the fundamental working principle of asymmetric encryption.